

Plant Disease Detection

End-to-End MLOps Application

Student: Ajaz Hussain
Roll Number: DA25M006
Course: DA5402 — Machine Learning Operations
Institution: IIT Madras
Submission: April 2026

GitHub: github.com/ajazhsn/Final_project

DagsHub: dagshub.com/ajazhsn/Final_project

1. Abstract

This report presents an end-to-end MLOps application for automated plant disease detection using deep learning. The system identifies six disease classes across potato and tomato plants using a custom Convolutional Neural Network achieving a **test macro F1 score of 0.9083**. The project implements the complete ML lifecycle — data versioning with DVC, experiment tracking with MLflow, model serving with FastAPI in Docker, a Streamlit frontend, orchestration with Apache Airflow, and real-time monitoring with Prometheus, Grafana, and AlertManager with Mailtrap email notifications.

2. Introduction

Plant diseases cause significant agricultural losses globally. Early and accurate detection is critical for timely intervention. Manual inspection is slow and expensive; automated deep learning systems offer a scalable alternative. This project builds a production-grade MLOps pipeline that takes a leaf image as input and returns a disease diagnosis with confidence score and treatment recommendation.

2.1 Objectives

- Train and compare three ML models: SVM, MLP, and Custom CNN
- Track experiments and register the champion model with MLflow
- Version data and pipeline stages with DVC + DagsHub
- Serve the model via FastAPI with Docker (3 replicas)
- Build a Streamlit frontend for end users
- Orchestrate batch predictions with Apache Airflow
- Monitor API health and performance with Prometheus and Grafana
- Deliver email alerts via AlertManager and Mailtrap SMTP

3. Dataset

The PlantVillage dataset was used, restricted to a 6-class subset covering potato and tomato diseases. Images were resized to 224×224 pixels and split 90/10 for train/validation with a separate held-out test set.

		Train	Validation	Test
Potato — Early blight	Disease	~830	~83	~55
Potato — Late blight	Disease	~830	~83	~55
Potato — Healthy	Healthy	~830	~83	~55
Tomato Early blight	Disease	~830	~83	~55
Tomato Late blight	Disease	~830	~83	~55
Tomato — Healthy	Healthy	~830	~83	~55
Total	—	4,991	996	664

Table 1: Dataset class distribution

3.1 Augmentation

Training images were augmented with random horizontal flips, rotations of up to 15 degrees, and colour jitter to improve generalisation. Augmentation was applied on-the-fly during training using PyTorch transforms.

4. Model Architecture and Training

4.1 Models Compared

Three models were trained and evaluated to select the best performer:

- **SVM with HOG features:** Classical baseline using histogram of oriented gradients
- **MLP:** Multi-layer perceptron on flattened pixel features
- **Custom CNN:** Three-block convolutional network (champion model)

	Training Accuracy	Validation Accuracy	Training Time
SVM + HOG	0.6132	0.6792	~37s
MLP	0.5843	0.6205	~3.2 min
Custom CNN (champion)	0.9083	0.9157	~13.4 min

Table 2: Model performance comparison

4.2 Custom CNN Architecture

The champion model consists of three convolutional blocks followed by fully connected layers:

Input (224×224×3) → Conv Block 1 (3→32) → Conv Block 2 (32→64) → Conv Block 3 (64→128) → AdaptiveAvgPool → FC(2048→256) → Dropout(0.3) → Output(6)

Each convolutional block contains Conv2d, BatchNorm2d, ReLU, and MaxPool2d. The network was trained with Adam (lr=0.0005), CrossEntropyLoss, for 25 epochs with batch size 32.

4.3 Per-Class F1 Scores (Champion Model)

	Early blight	Late blight	Healthy
Potato — Early blight	0.932	0.941	0.923
Potato — Late blight	0.867	0.878	0.856
Potato — Healthy	0.909	0.921	0.897
Tomato Early blight	0.841	0.855	0.827
Tomato Late blight	0.904	0.912	0.896
Tomato — Healthy	0.997	0.998	0.996

Macro Average	0.908	0.918	0.899
---------------	-------	-------	-------

Table 3: Per-class classification metrics

5. MLOps Pipeline

5.1 Data Versioning — DVC + DagsHub

All data is versioned with DVC and stored remotely on DagsHub. The pipeline defines four stages: prepare, train_svm, train_mlp, train_cnn, and evaluate. Running `dvc repro` reproduces the full experiment from raw data to metrics.

- Remote: `dagshub.com/ajazhsn/Final_project`
- Tags: `data-v0-raw`, `data-v1`, `data-v2`, `model-v2-champion`
- Metrics tracked: F1 score, accuracy, confusion matrix

5.2 Experiment Tracking — MLflow

All training runs are logged to MLflow with parameters, metrics, and artifacts. The champion CNN model is registered as `PlantDiseaseDetector@champion` (Version 2) in the MLflow Model Registry.

- Tracking URI: `sqlite:///mlflow.db`
- Registered model: `PlantDiseaseDetector@champion`
- Champion run ID: `18cb09fbc20e435c9613406d4f0d276a`

5.3 Model Serving — FastAPI + Docker

The champion model is served via FastAPI in a Docker container. Three replicas are deployed using Docker Compose for load distribution, exposing ports 8000, 8001, and 8002.

Endpoint	Method	Description
<code>/predict</code>	POST	Single image inference
<code>/predict/batch</code>	POST	Bulk image inference
<code>/health</code>	GET	Service health check
<code>/metrics</code>	GET	Prometheus metrics
<code>/classes</code>	GET	List supported classes

Table 4: FastAPI endpoint specifications

5.4 Frontend — Streamlit

A professional dark-themed Streamlit application provides single and bulk image detection, a pipeline console with live metrics, an MLOps tools page with clickable links to all services, and a service status dashboard.

- Single Detection: upload leaf image → instant diagnosis + treatment
- Bulk Detection: upload multiple images or ZIP file

- Pipeline Console: model performance, live API metrics, DB stats
- MLOps Tools: clickable links to MLflow, Grafana, Prometheus, Airflow

5.5 Orchestration — Apache Airflow

A FileSensor DAG monitors an input directory for CSV files containing image paths. When a file is detected, the pipeline runs batch predictions, stores results in SQLite, and sends email notifications via Mailtrap SMTP.

- DAG: plant_disease_pipeline
- Pool: scraper_pool (3 workers)
- Tasks: FileSensor → init_db → read_csv → batch_predict → check_failures → db_stats → notify → archive
- Email: Mailtrap SMTP (sandbox.smtp.mailtrap.io:587)

6. Monitoring and Alerting

6.1 Prometheus Metrics

Custom Prometheus metrics are exposed at /metrics on each FastAPI container. Prometheus scrapes all three backend replicas every 15 seconds.

Metric Name	Type	Description
plant_images_processed_total	Counter	Images processed by class/mode
plant_inference_latency_seconds	Histogram	Inference time distribution
plant_active_requests	Gauge	Current active requests
plant_prediction_confidence	Histogram	Confidence score distribution
plant_prediction_errors_total	Counter	Errors by type
plant_model_load_time_seconds	Gauge	Model load time at startup

Table 5: Custom Prometheus metrics

6.2 Grafana Dashboard

A Grafana dashboard visualises all metrics in real time with 5-second refresh. Panels include API health status, total images processed, request rate, CPU and memory usage, and inference latency percentiles (p50, p95, p99).

6.3 AlertManager + Mailtrap

AlertManager is configured with five alert rules. When triggered, alerts are routed to Mailtrap SMTP (port 587, STARTTLS) and delivered to the project email. Both FIRING and RESOLVED notifications were confirmed received.

Alert Rule	Threshold	Severity
HighErrorRate	> 10% error rate	Warning
HighInferenceLatency	p95 > 2 seconds	Warning
HighCPUUsage	> 80% for 5 min	Warning
APIDown	Any instance down	Critical
HighMemoryUsage	> 90% for 5 min	Critical

Table 6: AlertManager alert rules

6.4 Data Drift Monitoring

A pixel-statistics drift detector compares production image distributions against a training baseline. Features monitored include mean RGB values, standard deviations, brightness, and contrast. A 15% relative difference threshold triggers a retraining recommendation.

- Baseline computed from 300 training images
- Simulated production data showed 75% brightness drift
- Drift detected in: mean_r, mean_g, mean_b, brightness, contrast
- Action: engineer runs `mlflow run . --entry-point finetune_cnn`

7. Deployment Architecture

The entire stack is containerised with Docker Compose. All 11 services start with a single command:

```
docker compose up -d --scale backend=3
```

Service	Container	Port	Technology
Backend ×3	backend-1/2/3	8000–8002	FastAPI + uvicorn
Frontend	plant-frontend	8501	Streamlit
ML Tracking	plant-mlflow	5000	MLflow
Metrics	plant-prometheus	9090	Prometheus
Alerting	plant-alertmanager	9093	AlertManager
Dashboard	plant-grafana	3000	Grafana
Sys Metrics	plant-node-exporter	9100	node_exporter
Orchestration	plant-airflow-webserver	8080	Apache Airflow
Scheduler	plant-airflow-scheduler	—	Airflow Scheduler

Table 7: Docker Compose service manifest

7.1 Hardware Constraints and Design Decisions

The project was developed on a single machine with 7.6 GB RAM running WSL2 on Windows. Docker Swarm was not used due to memory constraints; instead, three Docker Compose replicas demonstrate equivalent orchestration. A custom CPU-only CNN was chosen over EfficientNet or ResNet as large pretrained models would take hours to train on CPU; the custom architecture achieved $F1 > 0.90$ in 13 minutes.

8. Testing

A pytest test suite covers all API endpoints with 19 test cases across four test classes. All tests pass against the live Docker container.

TestHealthEndpoint	5	Status, model loaded, classes, container ID
TestPredictEndpoint	7	200 response, correct class, confidence, probabilities, inference time, error handling
TestClassesEndpoint	4	6 classes, Potato and Tomato present
TestMetricsEndpoint	3	200 response, counter and histogram metrics
Total	19/19	All PASSED — 1.41s

Table 8: Unit test summary

9. Results Summary

All project components were implemented and verified:

Data versioning (DVC)	✓ PASS	5 DVC experiments on DagsHub
Experiment tracking	✓ PASS	MLflow runs, registered model
Model performance	✓ PASS	$F1=0.9083 > 0.90$ threshold
FastAPI backend	✓ PASS	3 Docker replicas, /health OK
Streamlit frontend	✓ PASS	Single + bulk detection working
Docker deployment	✓ PASS	11 containers via docker-compose
Airflow pipeline	✓ PASS	DAG proven, DB populated, emails sent
Prometheus metrics	✓ PASS	3/3 targets UP, custom metrics
Grafana dashboard	✓ PASS	10 panels, live data
AlertManager + email	✓ PASS	FIRING + RESOLVED emails in Mailtrap
Data drift monitoring	✓ PASS	75% brightness drift detected

Unit tests	✓ PASS	19/19 passing in 1.41s
------------	--------	------------------------

Table 9: Project deliverables completion status

10. Conclusions

This project successfully demonstrates a complete production-grade MLOps pipeline for plant disease detection. The custom CNN model achieves a test macro F1 of **0.9083**, significantly outperforming the SVM (0.6132) and MLP (0.5843) baselines. The full stack — from data versioning to monitoring and alerting — runs with a single Docker Compose command.

Key learnings include the importance of loose coupling between services (FastAPI and Streamlit communicate only via REST), the value of reproducible pipelines (DVC ensures any experiment can be reproduced exactly), and the criticality of monitoring in production (Prometheus caught the simulated APIDown condition and triggered email alerts within minutes).

Future work could include replacing SQLite with PostgreSQL for Airflow, implementing automated retraining when drift is detected, and extending the model to cover additional plant species and disease classes.

References

- PlantVillage Dataset — Penn State University
- MLflow Documentation — mlflow.org
- Apache Airflow Documentation — airflow.apache.org
- Prometheus Documentation — prometheus.io
- FastAPI Documentation — fastapi.tiangolo.com
- DVC Documentation — dvc.org